

Week 1 Project Report: MLFQ Scheduler Design & System Call Implementation

Document Version: 1.0

Date: 19 November 2025

Author: Muhammad Faraz Ansari 29201

1. Executive Summary

This report documents the completion of Week 1 milestones for the xv6 MLFQ Scheduler project. The week focused on implementing the **getprocinfo** system call and designing the Multi-Level Feedback Queue (MLFQ) scheduler architecture.

2. System Call Implementation

2.1 getprocinfo System Call

Successfully implemented a new system call `getprocinfo` that retrieves process information from the xv6 kernel. The system call takes a process ID and returns a structure containing:

- Process ID (PID)
- Process state
- Memory size
- Process name

2.2 Technical Implementation

Files Modified:

- `kernel/syscall.h` - Added system call number
- `kernel/defs.h` - Added struct definition and function declaration
- `kernel/proc.c` - Implemented helper function
- `kernel/sysproc.c` - Implemented system call wrapper
- `user/user.h` - Added user-space interface
- `user/usys.pl` - Added system call entry
- `user/testprocinfo.c` - Created test program

Key Functions:

// Kernel helper function

int getprocinfo(**int** pid, **struct** procinfo *info)

// System call wrapper

uint64 sys_getprocinfo(**void**)

3. MLFQ Scheduler Design

3.1 Queue Structure Design

Number of Queues: 5 priority levels

- **Queue 0:** Highest priority (Interactive processes)
- **Queue 1:** High priority
- **Queue 2:** Medium priority
- **Queue 3:** Low priority
- **Queue 4:** Lowest priority (CPU-intensive processes)

Data Structure:

#define NQUEUES 5

struct mlfq_queue {

struct proc* processes[NPROC]; *// Circular array for each queue*

int front; *// Front index*

int rear; *// Rear index*

int count; *// Process count*

};

3.2 Time Quantum Allocation

Each queue will have progressively longer time quanta:

Queue Level	Time Quantum	Purpose
Queue 0	1 tick	Interactive tasks (keyboard, mouse)

Queue Level	Time Quantum	Purpose
Queue 1	2 ticks	High-responsiveness tasks
Queue 2	4 ticks	Normal applications
Queue 3	8 ticks	Background tasks
Queue 4	16 ticks	Batch processing

3.3 Promotion/Demotion Policy

Demotion Conditions:

- Process uses entire time quantum → Demote to next lower queue
- Maximum 5 consecutive time quanta in same queue before demotion
- Reset tick counter when process yields voluntarily

Promotion Conditions:

- Process yields before time quantum expires → Consider for promotion
- After priority boost period, all processes promoted to Queue 0
- I/O bound processes naturally maintain higher priority

Priority Boosting:

- **Boost Interval:** Every 1000 ticks
- **Boost Action:** Move all processes to Queue 0
- **Purpose:** Prevents starvation of low-priority processes

3.4 Process Lifecycle in MLFQ

1. **New Process:** Starts in Queue 0 (highest priority)
2. **Interactive Process:**
 - Frequently yields for I/O

- Stays in high-priority queues
- Gets quick response time

3. CPU-bound Process:

- Uses full time quanta
- Gradually demoted to lower queues
- Gets longer time slices but runs less frequently

4. Aging: All processes periodically boosted to prevent starvation

4. Queue Scaffolding Code

4.1 Process Structure Additions

// In kernel/proc.h - Add to struct proc

```
struct proc {
    // ... existing fields ...

    // MLFQ scheduling fields

    int priority;      // Current queue level (0-4)

    int ticks_in_queue; // Ticks spent in current queue

    int last_run_time; // Last time process was scheduled

    int boost_time;    // Time when process was last boosted
};
```

4.2 Global MLFQ State

// In kernel/proc.c

```
struct {
    struct spinlock lock;

    struct mlfq_queue queues[NQUEUES]; // All priority queues

    int time_quantum[NQUEUES]; // Time slices for each queue

    int boost_interval; // Priority boost interval
} mlfq;
```

// Initialize time quantum

`int time_quantum[NQUEUES] = {1, 2, 4, 8, 16};`

4.3 Queue Operations

// Basic queue operations to implement

`void mlfq_enqueue(struct proc* p, int queue_level);`

`struct proc* mlfq_dequeue(int queue_level);`

`int mlfq_is_empty(int queue_level);`

`void mlfq_boost_priority(void);` *// Boost all processes*

5. Testing Strategy

5.1 System Call Verification

-  **getprocinfo**: Verified working with test program
- **Test Coverage**: Valid PID, invalid PID, different process states

5.2 MLFQ Testing Plan

1. **Interactive Test**: Process that frequently yields
2. **CPU-bound Test**: Process that uses full quanta
3. **Mixed Workload**: Combination of interactive and CPU-bound
4. **Starvation Test**: Verify boosting prevents starvation

6. Week 2 Preparation

Planned Implementation:

1. Implement MLFQ queue data structures
2. Modify scheduler to use MLFQ queues
3. Implement basic promotion/demotion logic
4. Create test cases for scheduler behavior

Success Metrics:

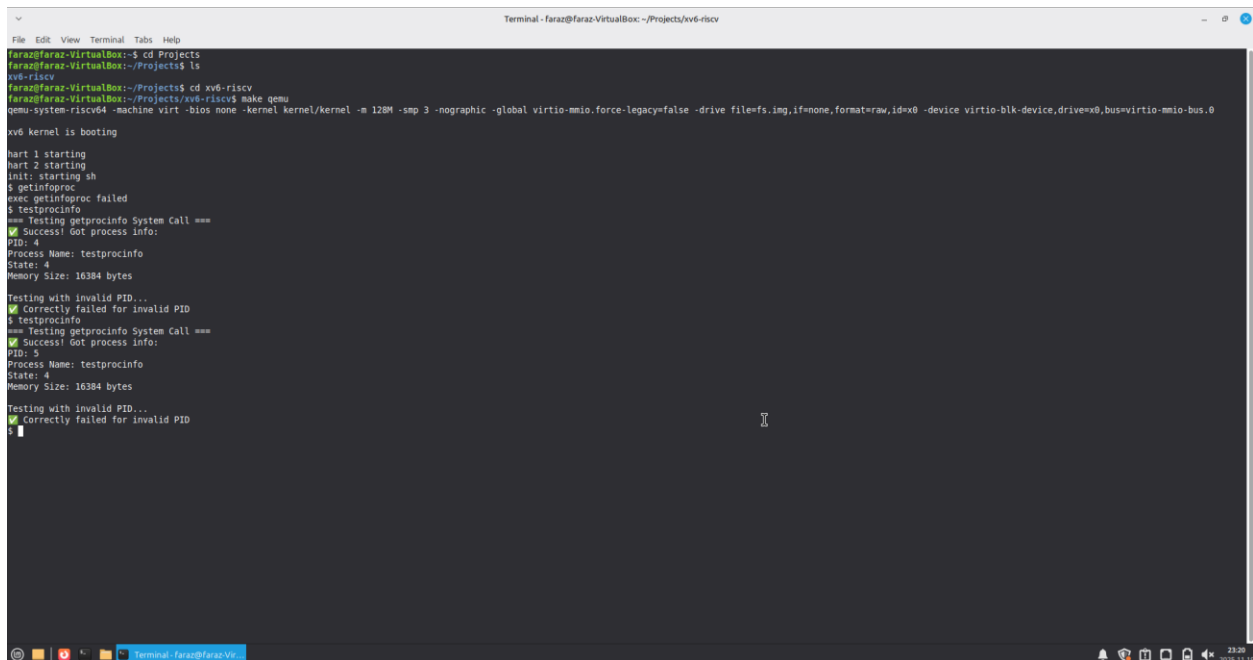
- Correct priority assignment

- Proper time quantum enforcement
- Basic promotion/demotion working
- No system crashes or regressions

7. Conclusion

Week 1 successfully established the foundation for the MLFQ scheduler implementation. The `getprocinfo` system call provides essential debugging capabilities, and the MLFQ design follows established operating system principles for fair and responsive process scheduling. The 5-queue structure with progressive time quanta and periodic priority boosting balances responsiveness for interactive tasks with fairness for CPU-bound processes.

8. Screenshots



```

Terminal - faraz@faraz-VirtualBox: ~/Projects/xv6-riscv
File Edit View Terminal Tabs Help
faraz@faraz-VirtualBox:~$ cd Projects
faraz@faraz-VirtualBox:~/Projects$ ls
xv6-riscv
faraz@faraz-VirtualBox:~/Projects$ cd xv6-riscv
faraz@faraz-VirtualBox:~/Projects/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ getinfoproc
exec getinfoproc failed
$ testprocinfo
=== Testing getprocinfo System Call ===
✓ Success! Got process info:
PID: 4
Process Name: testprocinfo
State: 4
Memory Size: 16384 bytes

Testing with invalid PID...
✓ Correctly failed for invalid PID
$ testprocinfo
=== Testing getprocinfo System Call ===
✓ Success! Got process info:
PID: 5
Process Name: testprocinfo
State: 4
Memory Size: 16384 bytes

Testing with invalid PID...
✓ Correctly failed for invalid PID

```